

ROS Parameter Server

Parameter Server jest zbiorem wartości parametrów odczytywanych przez nody w ROS. Parametry mogą przechowywać informacje o konfiguracji sprzętowej lub innych charakterystycznych parametrach. Serwer parametrów wykorzystuje 1 do 1 odpowiednik formatu YAML.

Dostępne typy parametrów:

- 32-bit integers
- booleans
- strings
- doubles
- iso8601 dates
- lists
- base64-encoded binary data

yaml przykłady plików konfiguracyjnych

uproszczony

```
string: 'jakis tekst'
integer: 3948
float: 231.5
boolean: false
list: [1.0, mixed list]
dictionary: {a: b, c: d}
```

dla lokalnej mapy kosztu

```
local_costmap:

  footprint: [[-0.205, -0.155], [-0.205,0.155], [0.077, 0.155], [0.077, -0.155]]

  global_frame: map # tb3_0/odom
  robot_base_frame: default_robot/base_footprint
  static_map: false
  rolling_window: true

  update_frequency: 10.0
  publish_frequency: 10.0
  transform_tolerance: 0.5

  static_map: false
  rolling_window: true
  width: 3
  height: 3
  resolution: 0.05

  plugins:
    - {name: obstacle_layer, type: "costmap_2d::ObstacleLayer"}
    - {name: inflation, type: "costmap_2d::InflationLayer"}
  obstacle_layer:
    observation_sources: laser
    laser: {data_type: LaserScan, sensor_frame: default_robot/base_scan, topic: /default_robot/scan, marking: true, clearing: true, obstacle_range: 2.0, raytrace_range: 5.0}

  inflation:
    inflation_radius: 0.75
    cost_scaling_factor: 4 # larger number = lower cost elevation
```

Istnieje możliwość zmiany pojedynczych parametrów, ale można je też nadpisać słownikiem w rosparam. Dla kątów w reprezentacji stopnie/radiany możliwe jest zastosowanie wyrażeń pozwalających na konwersję wartości. Dopuszczalne jest

użycie **pi**, które stanowi reprezentację liczby **PI**.

```
angle1: rad(3*pi)
angle2: !radians 3.14169
```

rosparam - komenda do obsługi parametrów

```
In [8]: # Komendy dla polecenia rosparam
!rosparam --help
```

rosparam is a command-line tool for getting, setting, and deleting parameters from the ROS Parameter Server.

Commands:

```
rosparam set      set parameter
rosparam get      get parameter
rosparam load     load parameters from file
rosparam dump     dump parameters to file
rosparam delete   delete parameter
rosparam list     list parameter names
```

Wyświetlanie listy parametrów:

rosparam list

Po uruchomieniu symulacji turtlesim i wyświetleniu listy parametrów dostępne są następujące parametry:

```
In [1]: !rosparam list
```

```
/rostdistro
/roslaunch/uris/host_localhost__37221
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

Dla samej symulacji domyślnie dostępne są tylko parametry odpowiedzialne za ustawienie koloru tła. Znajdują się one w przestrzeni nazw turtlesim (background_b, background_g, background_r).

Odczyt wartości parametru:

rosmmsg show *nazwa_parametru*

Przy odczycie nazwy całego parametru zwracana jest ustawiona wartość dla tego parametru. Wyświetlenie parametrów z ustawionymi kolorami dla symulacji turtlesim.

```
In [3]: !rosparam get /turtlesim/background_b
```

255

```
In [4]: !rosparam get /turtlesim/background_g
```

86

```
In [5]: !rosparam get /turtlesim/background_r
```

69

Możliwe jest odczytanie wartości wszystkich parametrów znajdujących się w danej przestrzeni nazw. Wyświetlenie wszystkich wartości składowych dla tła symulacji z przestrzeni nazw **turtlesim**. Wyświetlone zostaną te same wartości co poprzednio, ale z użyciem jednego polecenia.

```
In [6]: !rosparam get /turtlesim
```

```
background_b: 255
background_g: 86
background_r: 69
```

Ustawianie lub zmiana wartości parametru

rosparam set *nazwa_parametru* wartość

Dla parametrów, które nie istnieją w systemie dodawane są nowe o podanej nazwie. W przypadku parametrów istniejących ich wartość jest zmieniana. Dodanie nieistniejącego parametru **robot_1/tryb_pracy** o wartości **auto**.

```
In [7]: !rosparam set robot_1/tryb_pracy "auto"
```

```
In [8]: # na liście parametrów pojawił się parametr /robot_1/tryb_pracy
!rosparam list
```

```
/robot_1/tryb_pracy
/rosdistro
/roslaunch/uris/host_localhost__37221
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

```
In [9]: # wartość tego parametru wynosi
!rosparam get /robot_1/tryb_pracy
```

```
auto
```

Można ustawić więcej parametrów w danej przestrzeni nazw. W tym celu możliwe jest przekazanie w wiadomości słownika.

```
In [10]: !rosparam set robot_2 "{v_max: 4, omega_max: 2}"
```

```
In [11]: # na liście parametrów pojawiły się parametry dla robot_2 o nazwie omega_max i v_max
!rosparam list
```

```
/robot_1/tryb_pracy
/robot_2/omega_max
/robot_2/v_max
/rosdistro
/roslaunch/uris/host_localhost__37221
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

Ustawienie nowych wartości koloru tła dla symulacji turtlesim

Kolor symulacji zostanie ustawiony na zielony.

1. Ustawienie parametrów koloru tła.

```
In [12]: # Wersja 1
!rosparam set turtlesim "{background_b: 0, background_g: 255, background_r: 0}"
```

```
In [33]: # Wersja 2
!rosparam set /turtlesim/background_b 0
!rosparam set /turtlesim/background_g 255
!rosparam set /turtlesim/background_r 0
```

2. Weryfikacja poprawności ustawienia parametrów.

```
In [13]: !rosparam get /turtlesim
```

```
background_b: 0
background_g: 255
background_r: 0
```

3. Aktualizacja koloru tła - zmiana koloru na zielony Ustawienie koloru tła możliwe jest po użyciu serwisu reset lub clear. Należy

uważać z serwisem reset, bo usunie wszystkie ścieżki i dodane roboty do symulacji. Dla serwisu clear tylko narysowane ścieżki przez roboty zostaną usunięte.

```
In [18]: # Wersja 1
!rosservice call /reset
```

```
In [ ]: # Wersja 2
!rosservice call /clear
```

Usuwanie istniejącego parametru

rosparam delete *nazwa_parametru*

Usunięcie parametru /robot_1/tryb_pracy

```
In [19]: !rosparam delete /robot_1/tryb_pracy
```

```
In [20]: # z listy parametrów został usunięty parametr /robot_1/tryb_pracy
!rosparam list
```

```
/robot_2/omega_max
/robot_2/v_max
/rosdistro
/roslaunch/uris/host_localhost__37221
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

Zapis ustawionych parametrów do pliku .yaml

rosparam dump *nazwa_pliku.yaml*

Domyślnie polecenie zapisuje wszystkie ustawione parametry do pliku.

```
In [23]: !rosparam dump dump.yaml
```

Jako argument można podać nazwę przestrzeni z parametrami i zapisać je do pliku.

rosparam dump *nazwa_pliku.yaml* przestrzeń_nazw

Zapis parametrów z kolorami tła do pliku dla przestrzeni nazw **turtlesim** w symulacji. Parametr **-v** wyświetla m.in. informację o zapisanych parametrach.

```
In [25]: !rosparam dump -v bg_color.yaml /turtlesim
```

```
dumping namespace [/turtlesim] to file [bg_color.yaml]
/turtlesim/background_r=255
/turtlesim/background_g=255
/turtlesim/background_b=0
```

import parametrów z pliku .yaml

rosparam load *nazwa_pliku.yaml*

Plik z **nowe_parametry.yaml** zawiera parametry konfiguracyjne dla robotów o nazwach *robot_3* i *robot_4*.

```
In [26]: !rosparam load nowe_parametry.yaml
```

```
In [27]: !rosparam list
```

```
/robot_2/omega_max
/robot_2/v_max
/robot_3/goal_tolerance
/robot_3/omega_max
/robot_3/tryb
/robot_3/v_max
/robot_4/goal_tolerance/angle
/robot_4/goal_tolerance/x
/robot_4/goal_tolerance/y
/robot_4/omega_max
/robot_4/tryb
/robot_4/v_max
/rosdistro
/roslaunch/uris/host_localhost__37221
/rosversion
```

Import parametrów z pliku można wykonać pod istniejącą lub nową przestrzeń nazw. Dla zapisanych wcześniej kolorów tła ustawionych dla symulacji wartości kolorów zostaną zapisane w przestrzeni */kolor_tla*.

rosparam load *nazwa_pliku.yaml* przestrzeń_nazw

```
In [28]: !rosparam load bg_color.yaml /kolor_tla
```

```
In [29]: !rosparam list
```

```
/kolor_tla/background_b
/kolor_tla/background_g
/kolor_tla/background_r
/robot_2/omega_max
/robot_2/v_max
/robot_3/goal_tolerance
/robot_3/omega_max
/robot_3/tryb
/robot_3/v_max
/robot_4/goal_tolerance/angle
/robot_4/goal_tolerance/x
/robot_4/goal_tolerance/y
/robot_4/omega_max
/robot_4/tryb
/robot_4/v_max
/rosdistro
/roslaunch/uris/host_localhost__37221
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

Python

Lista dostępnych parametrów

```
In [31]: import rospy
         rospy.get_param_names()
```

```
Out[31]:
```

```
['/run_id',  
 '/roslaunch/uric/host localhost 37221']
```

Odczyt wartości parametru

rospy.get_param(nazwa_parametru)

```
In [34]: # Odczyt ustawionych wartości parametrów dla koloru tła symulacji turtlesim  
print(rospy.get_param("turtlesim/background_r"))  
print(rospy.get_param("turtlesim/background_g"))  
print(rospy.get_param("turtlesim/background_b"))  
  
0  
255  
0
```

Ustawienie wartości parametru

rospy.set_param(nazwa_parametru, wartość_parametru)

Ustawienie nowych wartości składowych kolorów tła symulacji turtlesim, aby uzyskać kolor czerwony.

```
In [35]: # Wersja 1  
# ustawienie każdej wartości oddzielnie  
rospy.set_param("turtlesim/background_r", 255)  
rospy.set_param("turtlesim/background_g", 0)  
rospy.set_param("turtlesim/background_b", 0)  
  
In [43]: # Wersja 2  
# ustawienie wartości kolorów w przestrzeni nazw turtlesim,  
# przekazanie wartości jako słownik  
rospy.set_param("turtlesim", {"background_r": 255, "background_g": 0, "background_b": 0})
```

Aby wprowadzone zmiany były widoczne należy wywołać dodatkowo serwis /clear lub /reset.

```
In [44]: !rosservice call /clear
```

rosparam - alternatywna biblioteka do obsługi parametrów

Różnica w użyciu funkcji wynika z formatu w jakim ustawiana jest wartość parametru. Musi on być typu string.

```
In [45]: import rosparam
```

Ustawianie wartości pojedynczego parametru.

```
In [46]: # z biblioteki rospy  
rospy.set_param('robot_on', False)  
# z biblioteki rosparam  
rosparam.set_param('robot_on', "true")
```

Ustawienie wartości przekazanych jako słownik.

```
In [47]: # z biblioteki rospy  
rospy.set_param('gains', {"background_r": 255, 'background_g': 0, "background_b": 0})  
# z biblioteki rosparam  
rosparam.set_param('gains', "{ 'background_r': 255, 'background_g': 0, 'background_b': 0 }")
```

Obsługa grupy parametrów

Dodanie parametrów do określonej przestrzeni nazw.

Ustawianie kilku różnych parametrów w określonej przestrzeni nazw. Na przykładzie kolorów RGB tła można by ustawić je w przestrzeni nazw **color** i zgrupować razem. Takie ustawienie parametrów nie będzie miało wpływu na środowisko symulacyjne (turtlesim), gdyż nie są one obsługiwane.

```
In [53]: grupa = "color"
rospy.set_param(grupa+"/r", 41)
rospy.set_param(grupa+"/g", 231)
rospy.set_param(grupa+"/b", 0)
```

Odczyt wartości parametrów z grupy color

```
In [54]: rospy.get_param("color")
```

```
Out[54]: {'r': 41, 'g': 231, 'b': 0}
```

Zapis parametrów do oddzielnych zmiennych ze składowymi kolorów.

```
In [55]: colors = rospy.get_param("color")
r, g, b = colors["r"], colors["g"], colors["b"]
print("Składowe kolorów r={}, g={}, b={}".format(r,g,b))
```

Składowe kolorów r=41, g=231, b=0

Sprawdzanie czy parametr istnieje

rospy.has_param(nazwa_parametru)

```
In [52]: rospy.has_param('robot_on')
```

```
Out[52]: True
```

```
In [53]: rospy.has_param('ten_parametr_nie_istnieje')
```

```
Out[53]: False
```

Usuwanie parametrów

rospy.delete_param(nazwa_parametru)

Do usunięcia pojedynczego parametru należy podać jego pełną nazwę.

```
In [63]: rospy.delete_param("/color/r")
```

Można usunąć więcej parametrów znajdującej się w określonej przestrzeni nazw.

```
In [64]: rospy.delete_param("/color/")
```

W trakcie próby usunięcia nieistniejącego parametru pojawia się błąd

```
In [65]: rospy.delete_param("/color/")
```

```
-----
KeyError                                Traceback (most recent call last)
/tmp/ipykernel_995/569920000.py in <module>
----> 1 rospy.delete_param("/color/")

/opt/ros/noetic/lib/python3/dist-packages/rospy/client.py in delete_param(param_name)
    566     """
    567     _init_param_server()
--> 568     del _param_server[param_name] #MasterProxy does all the magic for us
    569
    570 def has_param(param_name):

/opt/ros/noetic/lib/python3/dist-packages/rospy/msproxy.py in __delitem__(self, key)
    184     code, msg, _ = self.target.deleteParam(rospy.names.get_caller_id(), resolved_key)
    185     if code == -1:
--> 186         raise KeyError(key)
    187     elif code != 1:
    188         raise rospy.exceptions.ROSException("cannot delete parameter: %s"%msg)

KeyError: '/color/'
```

Pojawił się błąd `KeyError`, aby uniknąć nagłego przerwania działania programu można obsłużyć wyjątek z wykorzystaniem składni **try except**. Błąd typu **KeyError**, który pojawia się w bloku **try** jest obsługiwany w części **except** i pojawia się informacja "Parametr nie istnieje.". Nie pojawia się już błąd tak jak w powyższym przykładzie.

```
In [56]: try:
        rospy.delete_param('to_delete')
except KeyError:
    print("Parametr nie istnieje.")
```

Parametr nie istnieje.

Wyszukiwanie parametrów po nazwie klucza

Sprawdzanie czy parametr istnieje

rospy.search_param(nazwa_parametru)

```
In [81]: # Aktualna lista parametrów
!rosparam list

/background_b
/color/b
/color/g
/color/r
/gains/background_b
/gains/background_g
/gains/background_r
/kolor_tla/background_b
/kolor_tla/background_g
/kolor_tla/background_r
/robot_2/omega_max
/robot_2/v_max
/robot_3/goal_tolerance
/robot_3/omega_max
/robot_3/tryb
/robot_3/v_max
/robot_4/goal_tolerance/angle
/robot_4/goal_tolerance/x
/robot_4/goal_tolerance/y
/robot_4/omega_max
/robot_4/tryb
/robot_4/v_max
/robot_on
/rosdistro
/roslaunch/uris/host_localhost__37221
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

Dla parametru istniejącego na liście

```
In [78]: param_name = rospy.search_param('/robot_4/goal_tolerance/x')
v = rospy.get_param(param_name)

print("Nazwa parametru: {}".format(param_name))
print("Wartość parametru: {}".format(v))
```

Nazwa parametru: /robot_4/goal_tolerance/x
Wartość parametru: 0.1

Dla nieistniejącego parametru

```
In [79]: param_name = rospy.search_param('/robot_4/data')
v = rospy.get_param(param_name) # Nie znaleziono parametru, wyświetlane wszystkie parametry

print("Nazwa parametru: {}".format(param_name))
print("Wartość parametru: {}".format(v))
```


Nazwa parametru: None
Wartość parametru: {'run_id': '20ab98fc-c7bf-11ed-a99c-0242ac120002', 'roslaunch': {'uris': {'host_localhost_37221': 'http://localhost:37221/'}}, 'rosversion': '1.15.13\n', 'rostdistro': 'noetic\n', 'turtlesim': {'background_r': 255, 'background_g': 0, 'background_b': 0}, 'robot_1': {}, 'robot_2': {'v_max': 4, 'omega_max': 2}, 'robot_3': {'omega_max': 2, 'v_max': 4, 'goal_tolerance': 0.05, 'tryb': 'auto'}, 'robot_4': {'omega_max': 2, 'v_max': 4, 'goal_tolerance': {'x': 0.1, 'y': 0.05, 'angle': 3.141592653589793}, 'tryb': 'manual'}, 'kolor_tla': {'background_b': 0, 'background_g': 255, 'background_r': 255}}

Dla częściowej nazwy parametru

Nie wiadomo w jakiej przestrzeni nazw znajduje się dany parametr. Uruchomienie fragmentu programu, który występował by w przestrzeni nazw node'a spowodowałoby, że wyszukiwanie zaczęłoby się od tej przestrzeni. Poniższy program nie jest wywołany w żadnej przestrzeni nazw dlatego tylko znaleziony zostaje główny parametr **background_b**.

```
In [84]: param_name = rospy.search_param('background_b')
v = rospy.get_param(param_name)

print("Nazwa parametru: {}".format(param_name))
print("Wartość parametru: {}".format(v))
```

Nazwa parametru: /background_b
Wartość parametru: 255

Dla parametru **background_r** występuje on w różnych przestrzeniach nazw, ale poniższy kod nie jest wywołany pod żadną z nich, dlatego też nie zwraca informacji o znalezionej nazwie parametru.

```
In [86]: param_name = rospy.search_param('background_r')
v = rospy.get_param(param_name)

print("Nazwa parametru: {}".format(param_name))
print("Wartość parametru: {}".format(v))
```

Nazwa parametru: None
Wartość parametru: {'run_id': '20ab98fc-c7bf-11ed-a99c-0242ac120002', 'roslaunch': {'uris': {'host_localhost_37221': 'http://localhost:37221/'}}, 'rosversion': '1.15.13\n', 'rostdistro': 'noetic\n', 'turtlesim': {'background_r': 255, 'background_g': 0, 'background_b': 0}, 'robot_1': {}, 'robot_2': {'v_max': 4, 'omega_max': 2}, 'robot_3': {'omega_max': 2, 'v_max': 4, 'goal_tolerance': 0.05, 'tryb': 'auto'}, 'robot_4': {'omega_max': 2, 'v_max': 4, 'goal_tolerance': {'x': 0.1, 'y': 0.05, 'angle': 3.141592653589793}, 'tryb': 'manual'}, 'kolor_tla': {'background_b': 0, 'background_g': 255, 'background_r': 255}, 'robot_on': True, 'gains': {'background_r': 255, 'background_g': 0, 'background_b': 0}, 'color': {'r': 41, 'g': 231, 'b': 0}, 'background_b': 255, 'goal': {'tolerance': {'x': 0.05, 'y': 0.02}}, 'description': 'opis symulacji', 'robots': {'sim_robot2': {'v': 1.2, 'w': 0.3, 'type': 'heavy_transporter'}}, 'goal_tolerance': {'position': 0.1, 'orientation': 0.03}, 'turtle1': {'max_forward_vel': 1.2, 'max_rotation_vel': 0.3, 'goal_tolerance': 0.1, 'pen': {'r': 0, 'g': 0, 'b': 255, 'width': 5, 'off': False}}}

Dla wyszukiwania parametru w nodzie, który jest uruchomiony w przestrzeni nazw **/foo/bar** przeszukiwanie rozpoczyna się od nazw w tej przestrzeni i stopniowo przechodzi do coraz wyższych warstw. Dla parametru **param_example** wyszukiwanie będzie odbywało się w następującej kolejności:

1. /foo/bar/param_example
2. /foo/param_example
3. /param_example